An asymmetric verification framework for cloud-assisted RAG systems using Merkle mountain range auditing and dynamic boundary validation

Arulprasath V, Srinithi R, Sriram R, Jafer Sathik B and R. Senthil Prabha*

Department of Information Technology,
PSG College of Technology,
Coimbatore – 641 004, Tamil Nadu, India
Email: [author e-mail addresses to be supplied]
*Corresponding author

Abstract: Organisations increasingly outsource vector storage and similarity search for retrieval-augmented generation (RAG) to cloud providers they cannot fully trust. Such a provider can read stored chunks, forge or splice returned passages, silently skip index partitions, or expose one tenant's data to another. Zero-knowledge approaches make retrieval verifiable but impose server-side proving costs of several seconds. This paper presents a lightweight framework in which a trusted client verifies every cloud response through five sequential checks: role-based tenant authorisation, a two-tier Merkle mountain range (MMR) inclusion proof, dynamic boundary validation (DBV) of the probed Voronoi cells, AES-256-GCM authenticated decryption, and corrective-RAG grounding. Chunk payloads are encrypted before upload and committed to domain-separated MMRs. A Qdrant-based prototype is evaluated with component benchmarks, a DBV margin ablation and adversarial tests. Checking 22 retrieved chunks takes under 1 ms in-process, and ciphertext tampering, forged proofs, cell omission and cross-tenant access are rejected. The evaluation also exposes prototype gaps, which we report together with remedies.

Keywords: retrieval-augmented generation; cloud security; verifiable retrieval; Merkle mountain range; boundary validation; authenticated encryption; multi-tenant isolation; vector database.

Reference to this paper should be made as follows: Arulprasath, V., Srinithi, R., Sriram, R., Jafer Sathik, B. and Senthil Prabha, R. (xxxx) 'An asymmetric verification framework for cloud-assisted RAG systems using Merkle mountain range auditing and dynamic boundary validation', *Int. J. Cloud Computing*, Vol. x, No. x, pp.xxx–xxx.

Biographical notes: Arulprasath V, Srinithi R, Sriram R and Jafer Sathik B are B.Tech students in the Department of Information Technology, PSG College of Technology, Coimbatore. R. Senthil Prabha is a Professor in the same department. [Qualifications and research interests to be supplied.]

Copyright © xxxx Inderscience Enterprises Ltd.

1 Introduction

Large language models (LLMs) remain limited by static training data and a tendency to produce fluent but unsupported statements. Retrieval-augmented generation (RAG) reduces this problem by retrieving relevant passages from an external knowledge base and placing them in the model's context (Lewis et al., 2020). In a typical deployment, documents are split into chunks, embedded as dense vectors and indexed for approximate nearest-neighbour (ANN) search with structures such as inverted-file (IVF) partitions and hierarchical navigable small-world (HNSW) graphs (Johnson et al., 2021; Malkov and Yashunin, 2020). Because such indices need substantial memory and compute, many organisations hand vector storage and search to a cloud provider, which introduces a trust problem (Palanisamy et al., 2026).

The provider sees what it stores. Dense embeddings can be inverted to recover close approximations of the source text (Morris et al., 2023), and generated outputs can reveal whether a record is in the corpus (Li et al., 2025). Beyond confidentiality, the client has no native way to confirm that returned chunks are authentic, that the search covered the partitions it should have covered, or that results for one tenant were not drawn from another's data. A cost-saving provider can quietly skip partitions, and a compromised one can modify or splice passages to steer the LLM.

Verifiable-retrieval research addresses part of this problem with zero-knowledge proofs. VeriRAG (Lin et al., 2026), V3DB (Qiu et al., 2026), zkRAG (Jiang et al., 2026) and VERIF (Zhang et al., 2026) prove retrieval correctness to the client, but their authors report server-side proving times from several seconds to about a minute and a half per query on large corpora. These systems prove the whole retrieval computation, which is a stronger guarantee than the one pursued here.

This paper takes an *asymmetric* route. The client keeps a small amount of state, namely the encryption key, Merkle roots and IVF centroids, and performs inexpensive checks on what the cloud returns, so the cloud never generates a proof. The contributions are as follows.

- A two-phase architecture for tenant-isolated RAG on an untrusted vector engine, in which chunk payloads are AES-256-GCM encrypted and committed to a cryptographic audit structure before upload (Section 3).
- A two-tier Merkle mountain range (MMR) with domain-separated hashing that binds each chunk to its user's range and each user's root to an organisation root, with inclusion proofs of O(log N) hashes (Section 4.1).
- Dynamic boundary validation (DBV), a client-side computation of the IVF cells that a complete multi-probe search must cover, with a cluster-radius-scaled margin and an ablation of the margin factor (Sections 4.2 and 6.3).
- A five-check verifier that combines role-based access control, MMR proof checking, DBV, authenticated decryption and corrective-RAG grounding before any text reaches the LLM (Section 4.5).
- An evaluation of a working prototype that includes component benchmarks, attack tests run directly on the implementation, and a candid account of the guarantees the current prototype does and does not provide (Sections 6 and 7).

Section 2 reviews related work. Section 3 states the threat model and architecture, and Section 4 describes the verification mechanisms. Section 5 summarises the implementation, Section 6 reports the evaluation, Section 7 discusses limitations, and Section 8 concludes.

2 Related work

2.1 Cryptographic approaches to outsourced vector search

Four families of techniques are commonly applied to outsourced search. Order-preserving encryption supports range queries over scalar values but leaks ordering and cannot evaluate multi-dimensional similarity (Agrawal et al., 2004). Fully homomorphic encryption, for example the CKKS scheme (Cheon et al., 2017), computes on ciphertexts but at a large latency and storage cost for high-dimensional nearest-neighbour search. Hardware enclaves such as Intel SGX keep computation in protected memory, but they are exposed to side-channel attacks and enclave memory limits (Costan and Devadas, 2016). Zero-knowledge succinct arguments (Groth, 2016; Gabizon et al., 2019) let a server prove that a computation was done correctly, but expressing floating-point similarity and top-k selection as arithmetic circuits is expensive.

2.2 Privacy-preserving and verifiable RAG

ppRAG applies distance-preserving matrix encryption so that a cloud can run ANN search on transformed vectors without homomorphic encryption (Ye et al., 2026), but offers no check on integrity or search completeness. Membership-inference attacks against RAG (Li et al., 2025) and differentially private generation (Grislain, 2024) address leakage at the generation stage and, in the latter case, at some cost in answer accuracy. ZKSQL (Li et al., 2023) and ZKGraph (Wu et al., 2025) verify relational and graph queries in zero knowledge, but neither targets continuous similarity search. VeriRAG, V3DB, zkRAG and VERIF are the closest vector-retrieval systems: each reports its own proving and verification cost and memory use, summarised in Table 1. Palanisamy et al. (2026) survey RAG security and note that defences remain fragmented across confidentiality, integrity and availability.

Table 1 Related approaches and their reported characteristics (figures are as reported by the cited authors and were not reproduced).

Approach	*Core method*	*Main guarantee*	*Reported cost or limitation*
ppRAG (Ye et al., 2026)	Distance-preserving matrix encryption	Vector confidentiality against passive observers	No integrity or completeness check
S2MIA (Li et al., 2025)	Black-box membership inference	Attack analysis only	No defence architecture
DP-RAG (Grislain, 2024)	Differential-privacy noise at decoding	Bounded leakage per query	Accuracy loss; privacy budget depletes
ZKSQL (Li et al., 2023)	Zero-knowledge SQL evaluation	Correct relational queries	Not designed for similarity search
ZKGraph (Wu et al., 2025)	Expansion-centric ZK operators	Correct graph queries	General graph databases, not HNSW
VeriRAG (Lin et al., 2026)	Algorithm-specific ZK arguments	Verified top-k ANN result	Prover 96 s, verifier 3 s on 37 GB
V3DB (Qiu et al., 2026)	Audit-on-demand zk-SNARKs, IVF-PQ	Search bound to a committed snapshot	Proof per audit; snapshot management
zkRAG (Jiang et al., 2026)	Succinct ZK argument for HNSW	Correct HNSW retrieval	About 50 s per query (10^6 vectors, d = 128)
VERIF (Zhang et al., 2026)	ZK PIOP for IVF-Flat retrieval	Proof of full IVF-Flat retrieval	Proving 5.90–11.57 s; verification 0.62–1.48 s
This work	**AES-GCM, two-tier MMR, DBV, RBAC**	**Payload authenticity and confidentiality, cell-coverage check, tenant isolation**	**No proof of exact top-k; gaps listed in Table 9**

2.3 Authenticated data structures

Merkle trees authenticate a set of records with a single digest (Merkle, 1987). Crosby and Wallach (2009) developed history trees for tamper-evident logging. The Merkle mountain range, as described by Todd (2018), is an append-only collection of perfect binary trees whose peaks are combined into one root; completed subtrees never change, so proofs issued earlier remain valid when leaves are appended. Hashing uses SHA-256 (NIST, 2015), and payload protection uses AES-GCM authenticated encryption (Dworkin, 2007).

2.4 Gap addressed

Existing work covers confidentiality, integrity or completeness largely in isolation, and the zero-knowledge systems place the proving burden on the server. We study a combination that is cheap for the client to check: authenticated encryption, a hierarchical append-only audit structure for multi-tenant data, an IVF boundary check, and tenant-level access control, evaluated on a working prototype with its limitations stated.

3 Threat model and architecture

3.1 Threat model

The cloud vector engine is treated as honest-but-curious or actively malicious. It can read everything it stores, return modified, forged or re-assigned payloads, drop index partitions from a search, and serve results from the wrong tenant's data. The client service is trusted: it holds the AES-256 key, computes the MMR roots and holds the IVF centroids. Compromise of the client, hardware enclaves, side channels, denial of service and network-

level attacks are out of scope. Table 2 lists the threats considered, the mechanism that addresses each and its status in the prototype, and Table 3 shows where each asset resides.

Table 2 Threats, mechanisms and implementation status.

Threat	*Adversarial action*	*Mechanism*	*Status in prototype*
T1 Plaintext exposure	Read chunk text held by the cloud	AES-256-GCM encryption before upload	Implemented
T2 Embedding inversion	Invert stored vectors to text	Vector masking (orthogonal transform)	Not in prototype; vectors stored unmasked
T3 Chunk forgery or splicing	Modify or substitute a returned chunk	MMR inclusion proof; GCM tag	Implemented, with gaps (Table 9)
T4 Cell omission	Skip IVF cells during search	DBV set-inclusion check	Implemented as a set check
T5 Cross-tenant access	Query another organisation's data	Per-tenant collections; RBAC check	Implemented at the client service
T6 Ungrounded generation	Irrelevant or poisoned context	CRAG grading; Self-RAG tokens; grounding score	Implemented as heuristics

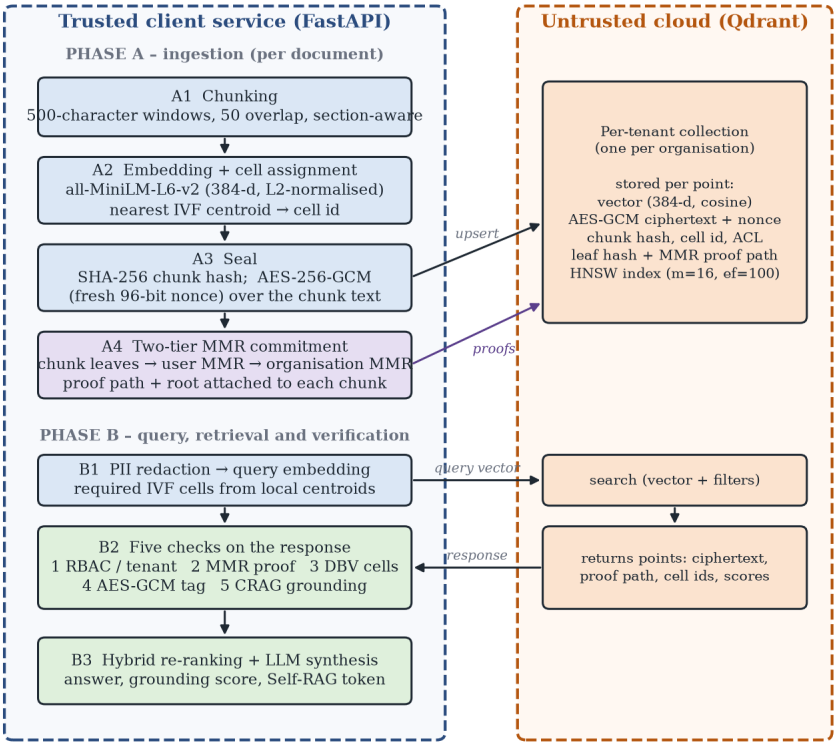
Table 3 Assets and trust domains in the prototype.

Asset	*Trusted client service*	*Untrusted cloud engine*
Chunk plaintext	Read after authenticated decryption	Ciphertext only
AES-256 key	Held in service configuration	Never sent
Chunk embeddings (384-d)	Generated locally	Stored unmasked (cosine search needs them)
Chunk hash, nonce, ACL, cell id	Created at ingestion	Stored as point payload
MMR proof path and root	Computed at ingestion; root recorded	Stored as point payload
IVF centroids and radii	Trained and stored locally	Not needed (filters by cell id)
User credentials	SQLite; PBKDF2-HMAC-SHA256, 100,000 iterations	Never sent

3.2 Architecture

Figure 1 shows the two phases. In Phase A the client service chunks a document, embeds each chunk, assigns it to an IVF cell, encrypts the text, hashes it, commits it to the two-tier MMR and uploads the result to a collection reserved for the tenant. In Phase B the client redacts personal identifiers from the query, embeds it, computes the cells a complete search must cover, fetches candidates from the cloud and applies the five checks before synthesis. In the prototype the client service and the Qdrant instance run on one machine, so the trust boundary is logical and network latency is absent from the measurements in Section 6.

Figure 1 Architecture of the framework: ingestion (Phase A) and verified query processing (Phase B) across the trusted client service and the untrusted vector engine. (see online version for colours)



3.3 Phase A: ingestion

Text is extracted from PDF or plain-text input and split into windows of 500 characters with a 50-character overlap, preferring section and sentence boundaries. Each chunk is embedded with all-MiniLM-L6-v2 (Reimers and Gurevych, 2019; Wang et al., 2020), which yields L2-normalised 384-dimensional vectors, and is assigned to the nearest IVF centroid by cosine distance. The chunk text is hashed with SHA-256 and encrypted with AES-256-GCM under a fresh 96-bit random nonce. Each tenant has its own collection, and each point carries an access-control list naming the owning user and the organisation administrator. The chunks of a user are then committed to a user MMR whose root becomes a leaf of the organisation MMR (Section 4.1), and each point stores its leaf hash, proof path and the organisation root.

3.4 Phase B: query and retrieval

A regular-expression filter redacts e-mail addresses, national identifiers, phone numbers, card numbers and IP addresses from the query before embedding. The client then computes the required IVF cells (Section 4.2) and retrieves candidates from the tenant's collection. The prototype supports two retrieval paths: an exact-scoring path that reads every point of the tenant's collection, which the interactive application uses by default for the small collections in our deployment, and a multi-probe path that probes the required IVF cells and an HNSW nearest-neighbour search. Candidates then pass the five checks of Section 4.5.

4 Verification mechanisms

4.1 Two-tier Merkle mountain range

Every hash is SHA-256 with a one-byte domain prefix so that values from different levels cannot be confused. A chunk leaf commits to the chunk's position and to the hash of its plaintext, where $H(\cdot)$ denotes SHA-256 and $\|$ concatenation:

$$\text{leaf} = H(0\text{x}00 \,\|\, \text{org} \,\|\, \text{user} \,\|\, \text{doc} \,\|\, \text{chunk} \,\|\, \text{index} \,\|\, H(\text{plaintext})) \tag{1}$$

The identifiers are joined with a colon separator. An internal node hashes its two children, and the peaks $P_1, \ldots, P_m$ of a range are bagged from right to left into a single value:

$$\text{node}(a, b) = H(0\text{x}01 \,\|\, a \,\|\, b) \tag{2}$$

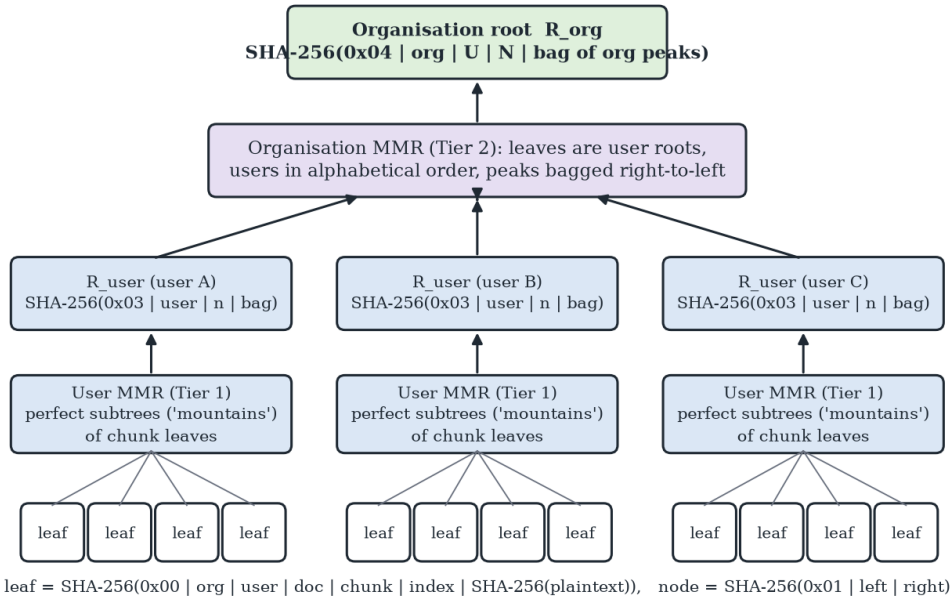
$$B_m = P_m, \quad B_i = \text{node}(P_i, B_{i+1}), \quad \text{bag} = B_1 \tag{3}$$

A user MMR (Tier 1) holds the leaves of one user's chunks, sorted by document, index and chunk identifier. Its root includes the user and the chunk count. The organisation MMR (Tier 2) takes the user roots, in alphabetical order of user identifier, as its leaves, and its root includes the organisation, the number of users U and the total number of chunks N:

$$R_{\text{user}} = H(0\text{x}03 \,\|\, \text{user} \,\|\, n_u \,\|\, \text{bag}_{\text{user}}) \tag{4}$$

$$R_{\text{org}} = H(0\text{x}04 \,\|\, \text{org} \,\|\, U \,\|\, N \,\|\, \text{bag}_{\text{org}}) \tag{5}$$

Figure 2 Two-tier MMR: chunk leaves are committed to per-user ranges, whose roots are leaves of the organisation range. (see online version for colours)



An inclusion proof lists, for the chunk's mountain, the sibling hashes with their left or right position, the list of user peaks, and the corresponding data for the organisation tier. Verification (i) recomputes the leaf from the chunk attributes, (ii) climbs the sibling path to the user peak, (iii) checks that the peak is in the peak list, bags the peaks and recomputes R_{user}, (iv) repeats steps (ii) and (iii) for the organisation tier with R_{user} as the leaf, and (v) compares the resulting root with the expected organisation root. A proof contains O(log n_u + log U) sibling hashes; Table 5 reports measured path lengths.

4.2 Dynamic boundary validation

The IVF index partitions the vector space into C cells around centroids $\mu_1, \ldots, \mu_C$ learned by k-means. With cosine distance $d(q, \mu) = 1 - \cos(q, \mu)$, let $d_i = d(q, \mu_i)$, let $i*$ be the nearest cell with $d_{\min} = d_{i*}$, and let r_i be the radius of cell i, defined as the largest cosine distance between a member of the cell and its centroid. The radii are stored with the centroids at indexing time. The client requires the nearest cell and every cell whose centroid is within a radius-scaled margin of it:

$$S_{\text{req}} = \{i*\} \cup \{i \neq i* : d_i - d_{\min} < \lambda \cdot r_{i*}\}, \quad \lambda = 1.5 \tag{6}$$

When $d_{\min} = 0$ only the nearest cell is required. A response is accepted when the cells reported as probed contain every required cell, and the coverage ratio $|S_{\text{req}} \cap S_{\text{ret}}| / |S_{\text{req}}|$ is recorded:

$$\text{accept} \;\Leftrightarrow\; S_{\text{req}} \subseteq S_{\text{ret}} \tag{7}$$

The margin is scaled by the cell radius, not by $d_{\min}$, because in 384 dimensions cosine distances to different centroids are close to one another and a ratio to a small $d_{\min}$ is unstable. DBV checks *compliance* with the agreed set of cells; it does not prove that the exact global top-k was returned, and it applies only to IVF-

partitioned search (HNSW has no cells). In the prototype, the cells reported by the cloud are taken from the cell identifiers stored in the returned points. Those identifiers are not part of the MMR leaf, so DBV in its present form defends against a lazy cloud, not against one that reports false identifiers (Section 7).

4.3 Authenticated encryption and access control

Chunk text is encrypted with AES-256-GCM using a random 96-bit nonce, which produces a ciphertext with a 128-bit authentication tag (Dworkin, 2007). Decryption fails if the ciphertext or tag has been altered, so a modified ciphertext never yields plaintext. Access control uses a registry of users with an organisation, a role and an approval status. Passwords are stored with PBKDF2-HMAC-SHA256 (Moriarty et al., 2017) at 100,000 iterations, and sign-in with a Google identity token is also supported. A request is authorised only if the user exists, is approved, and belongs to the organisation that owns the target collection, after organisation aliases are normalised.

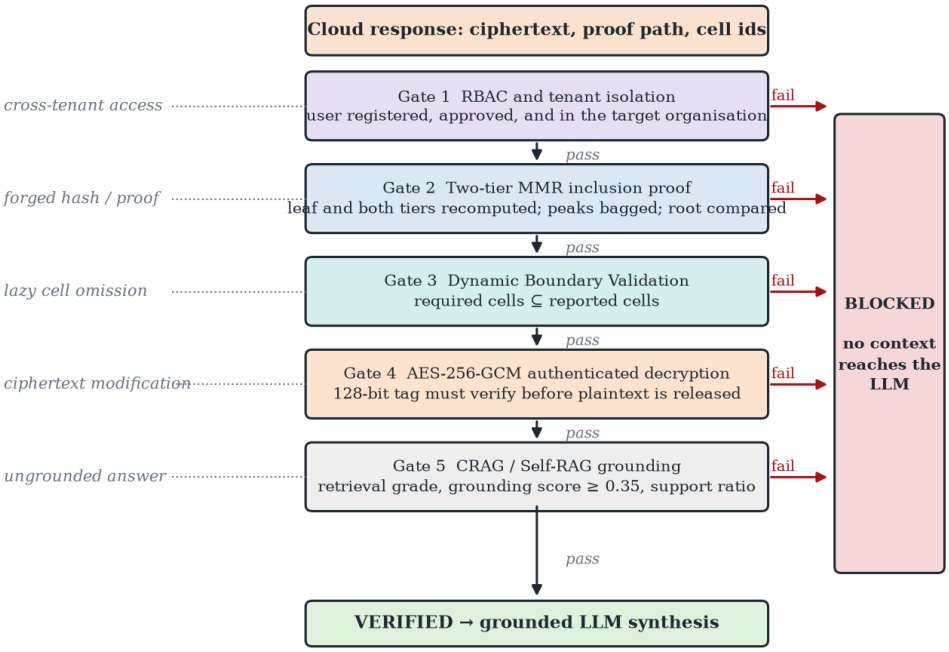
4.4 Retrieval quality and grounding

Decrypted candidates are scored by a hybrid of dense cosine similarity and BM25 lexical scores (Robertson and Zaragoza, 2009), fused by reciprocal rank fusion (Cormack et al., 2009) and optionally re-ranked by a MiniLM cross-encoder, with a lexical fallback when the model is unavailable. A corrective-RAG-style grader (Yan et al., 2024) blends retrieval score and query-term overlap into a confidence and labels the retrieval correct (≥ 0.55), ambiguous (≥ 0.25) or incorrect. After generation, a grounding score compares the answer with the context, with a threshold of 0.35, and a Self-RAG-style reflection (Asai et al., 2024) reports a support ratio and a reflection token. These components are rule-based approximations and not trained critics.

4.5 The five checks

Figure 3 shows the verification sequence applied to a cloud response. Gate 1 checks authorisation, Gate 2 verifies MMR proofs, Gate 3 applies DBV, Gate 4 decrypts with tag verification, and Gate 5 applies grounding. A failure at any gate blocks generation. Gate 1 is also the point at which cross-tenant requests are rejected, and the response then reports zero chunks.

Figure 3 The five checks applied to a cloud response and the threat each one addresses. (see online version for colours)



Security analysis. *Gate 1* blocks a user who is unregistered, pending, rejected or in another organisation. *Gate 2* detects any change to a chunk's identifiers or plaintext hash and any change to a sibling hash, because the recomputed root then differs. It does not cover the ciphertext itself, which the leaf does not include. *Gate 3*

detects omission of a required cell from the reported set. *Gate 4* detects modification of ciphertext, nonce or tag. *Gate 5* filters irrelevant context but offers no cryptographic guarantee. Section 6.4 tests each claim on the implementation and reports two cases that the present checks do not catch.

5 Implementation

The prototype is written in Python and exposes a FastAPI service with endpoints for ingestion, verified query, retrieval and administration, and a browser front end. Table 4 lists the main components and parameters.

Table 4 Implementation specification.

Component	*Specification*
Service	Python, FastAPI, Pydantic; SQLite for users and approvals
Vector engine	Qdrant v1.19.1, one cosine collection per tenant, 384 dimensions; HNSW m = 16, ef_construct = 100
Embedding	all-MiniLM-L6-v2 (384-d, normalised); hashed bag-of-words fallback when the model is unavailable
Chunking	500-character windows, 50-character overlap, section-aware
Cryptography	AES-256-GCM (96-bit random nonce, 128-bit tag); SHA-256; PBKDF2-HMAC-SHA256 (100,000 iterations)
Audit structure	Two-tier MMR, domain prefixes 0x00–0x04, peaks bagged right to left
IVF boundary	Six cells by default (k-means), cell radii stored with centroids, $\lambda = 1.5$
Re-ranking	Dense + BM25 with reciprocal rank fusion; MiniLM cross-encoder with heuristic fallback; LangChain multi-query retrieval with FlashRank
Generation	External LLM (Gemini or OpenAI) when keys are configured; local extractive synthesiser otherwise

6 Experimental evaluation

6.1 Setup

All measurements were taken on a Windows 11 machine with an Intel x86-64 CPU, using Python 3.13.12 in a single process with the Qdrant layer replaced by in-memory structures where noted. Timings are medians over repeated in-process runs and exclude network transfer and LLM calls. The component and attack tests call the project's own hashing, MMR, encryption, boundary-validation and authorisation modules directly.

The ablation uses a corpus of 188 chunks (66,237 characters) produced by the project's chunker from the text of the project report, and 200 queries, each a sentence drawn from a chunk. The ground truth for a query is the exact top-5 chunks by cosine similarity among all other chunks, so the source chunk is excluded. The embedding model weights were not available in the benchmark environment, so the prototype's deterministic hashed bag-of-words embedder (384-d) was used, and k-means was run with an equivalent NumPy implementation. The ablation figures therefore describe this embedder and corpus; the experiment script in the repository reruns them unchanged with the neural encoder.

6.2 Component cost

Table 5 Median client-side cost of the verification components (in-process, single thread).

Component	*Setting*	*Median (ms)*	*95th percentile (ms)*
Two-tier MMR proof verification	100 chunks, 10 users; mean 5.2 sibling hashes	0.010	0.013
Two-tier MMR proof verification	1,000 chunks, 10 users; mean 7.8 sibling hashes	0.014	0.022
Two-tier MMR proof verification	10,000 chunks, 10 users; mean 11.04 sibling hashes	0.020	0.025
AES-256-GCM decryption	500-byte chunk	0.0044	0.0050
DBV check	6 cells, 384-d query	0.044	0.057
Gates 2, 3 and 4 for one query	5 retrieved chunks	0.125	0.230
Gates 2, 3 and 4 for one query	22 retrieved chunks	0.351	0.551
Gates 2, 3 and 4 for one query	50 retrieved chunks	0.761	1.236

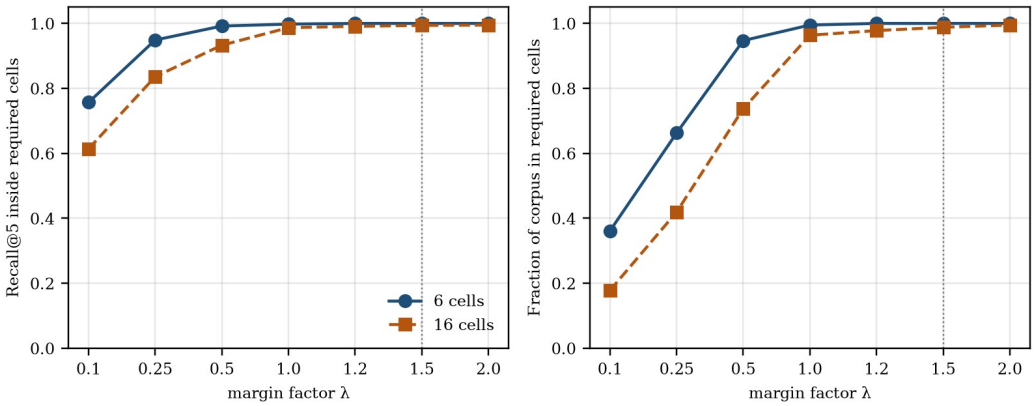
Verification of a single proof takes 10 µs at 100 chunks and 20 µs at 10,000 chunks, and the proof grows from 5.2 to 11.04 sibling hashes, consistent with logarithmic growth. Verifying the proofs, tags and cells for 22 retrieved chunks takes 0.35 ms. These figures measure the client's checking work only. They omit network transfer, embedding, re-ranking and generation, and they are not comparable with server-side proving times reported for zero-knowledge systems, which do a different job.

6.3 Dynamic boundary validation ablation

Table 6 Margin factor λ: recall of cell-restricted search, required cells and omission detection (200 queries, 188 chunks; hashed bag-of-words embedder).

IVF cells	*λ*	*Recall@5*	*Mean required cells*	*Corpus in required cells*	*Queries needing ≥ 2 cells*	*Omissions detected*
6	0.1	0.757	1.63	36.1%	38.5%	326/326
6	0.25	0.949	3.33	66.3%	78.5%	667/667
6	0.5	0.992	5.54	94.7%	98.5%	1107/1107
6	1	0.998	5.97	99.5%	99.5%	1195/1195
6	1.2	1.000	6.00	100.0%	100.0%	1200/1200
6	1.5	1.000	6.00	100.0%	100.0%	1200/1200
6	2	1.000	6.00	100.0%	100.0%	1200/1200
16	0.1	0.614	1.69	17.8%	37.0%	337/337
16	0.25	0.836	4.46	41.9%	75.5%	891/891
16	0.5	0.933	10.43	73.8%	89.5%	2086/2086
16	1	0.987	15.29	96.4%	98.5%	3058/3058
16	1.2	0.991	15.60	97.8%	99.0%	3119/3119
16	1.5	0.994	15.80	98.8%	99.5%	3160/3160
16	2	0.995	15.92	99.5%	99.5%	3184/3184

Figure 4 Effect of the margin factor on recall@5 and on the fraction of the corpus that must be probed (dotted line: λ = 1.5). (see online version for colours)



The margin factor controls a trade-off between audit strictness and the share of the index that has to be probed. With six cells, λ = 0.1 requires 1.63 cells on average and restricts search to 36% of the corpus, but only 76% of the true top-5 chunks fall inside those cells. Raising λ to 0.5 recovers 99.2% recall at 95% of the corpus, and at the adopted value λ = 1.5 recall is 1.000 (six cells) and 0.994 (sixteen cells). At that setting, however, DBV requires 100% and 98.8% of the corpus, so on this corpus and embedder the check behaves close to a full-coverage requirement and saves little search work. A larger value of λ buys recall by widening the required set. Whether a smaller λ is preferable depends on the recall the application needs and on how the cells are distributed for the deployed encoder, which should be measured on the target corpus before λ is fixed.

Every injected omission was detected, as expected of a set-inclusion test: each required cell was dropped in turn from the reported set, and all 1,200 and 3,160 attempts at λ = 1.5 were rejected for six and sixteen cells respectively. Because the honest provider reports exactly the required set, the false-alarm rate on honest runs is zero by construction; the check does not measure how often a real, approximate engine would legitimately miss a cell.

6.4 Adversarial tests on the implementation

Table 7 Attack tests run against the implementation (function level).

Test	*Attack*	*Detecting check*	*Outcome*
A1	Flip one bit of a stored ciphertext	Gate 4 (GCM tag)	Rejected by the tag
A1b	Same attack, seen by the MMR proof	Gate 2	Not detected: the leaf commits to the plaintext hash only
A2	Replace the chunk hash in the proof	Gate 2 (MMR)	Rejected
A3	Modify one sibling hash in the proof path	Gate 2 (MMR)	Rejected
A4	Omit one required IVF cell	Gate 3 (DBV)	All attempts rejected (Table 6)
A5	User from another organisation queries a tenant	Gate 1 (RBAC)	Rejected
A5b	Pending or unknown user queries own organisation	Gate 1 (RBAC)	Rejected
A6	Swap ciphertext and nonce between two chunks	Gate 4	Not detected: decryption succeeds
A7	Forged chunk with its own self-consistent proof	Gate 2 with root from the response	Accepted
A7b	Same forgery checked against the trusted root	Gate 2 with client-held root	Rejected

Tests A1 to A5 confirm that each gate rejects the attack it was designed for. A1b, A6 and A7 show the limits of the present implementation. The MMR leaf commits to the hash of the plaintext and not to the ciphertext, so tampering with ciphertext is caught by the GCM tag at Gate 4 rather than by the MMR. AES-GCM is invoked without associated data, and the plaintext hash is not rechecked after decryption, so a ciphertext and nonce swapped between chunks that share the key decrypts successfully. Finally, the query path takes the expected organisation root from the proof that accompanies the response, so an attacker who builds a self-consistent proof for a forged chunk is accepted; the same proof is rejected when verified against a root held by the client (A7b). The remedies are listed in Table 9.

6.5 Operational validation

Figures 5 to 7 show the prototype in operation. Figure 5 shows the Qdrant dashboard with one collection per tenant, each standardised on 384-dimensional cosine vectors. Figure 6 shows a stored point, whose text is replaced by a Base64 ciphertext with its nonce, chunk hash and access-control list, and the leaf hash and two-tier proof path that accompany it. Figure 7 shows an authorised query returning a grounded answer and a cross-tenant query that is blocked.

Figure 5 Qdrant collections for five tenants; all report green health, 384-dimensional cosine vectors, six segments and one shard. (see online version for colours)

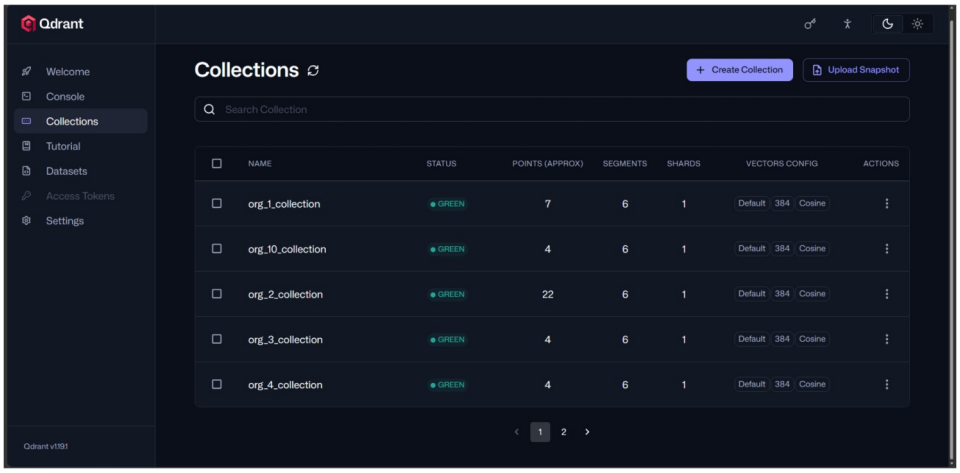
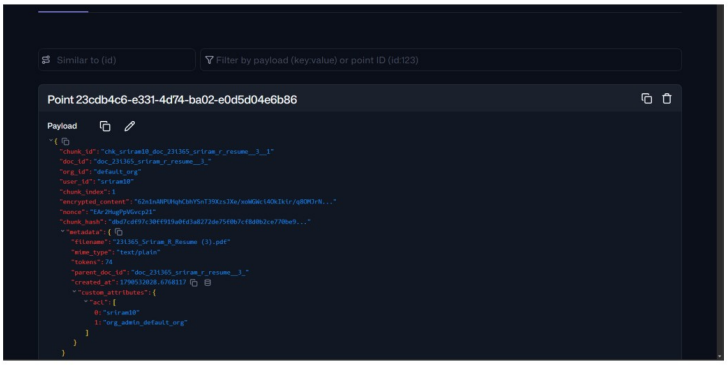


Figure 6 Stored point payload: (a) ciphertext, nonce, plaintext hash and access-control list; (b) leaf hash and two-tier proof path stored beside the vector. (see online version for colours)

(a) stored payload: ciphertext, nonce, hash, ACL



(b) leaf hash and two-tier proof path

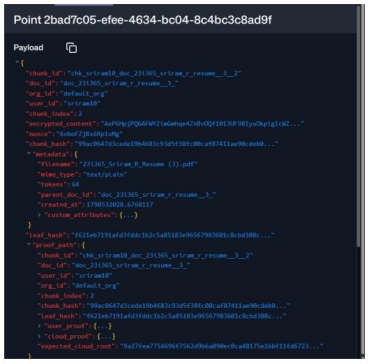
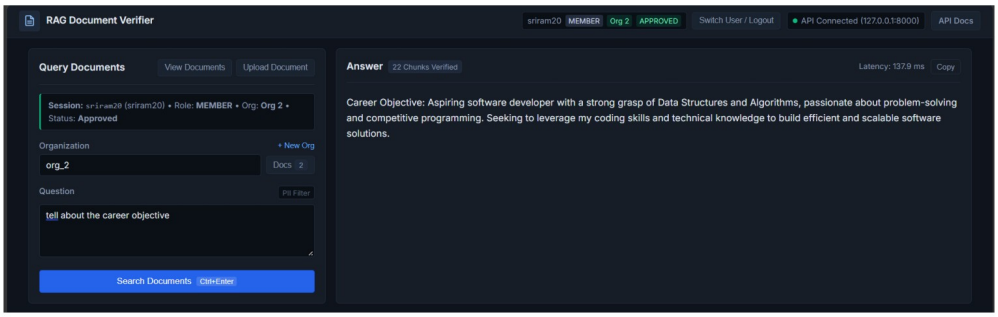


Figure 7 Verified query interface: (a) an approved member queries their own organisation; (b) a member of another organisation is blocked and no chunks are released. (see online version for colours)

(a) authorised query: 22 chunks verified, 137.9 ms



(b) cross-tenant query blocked, 0 chunks released, 137.64 ms

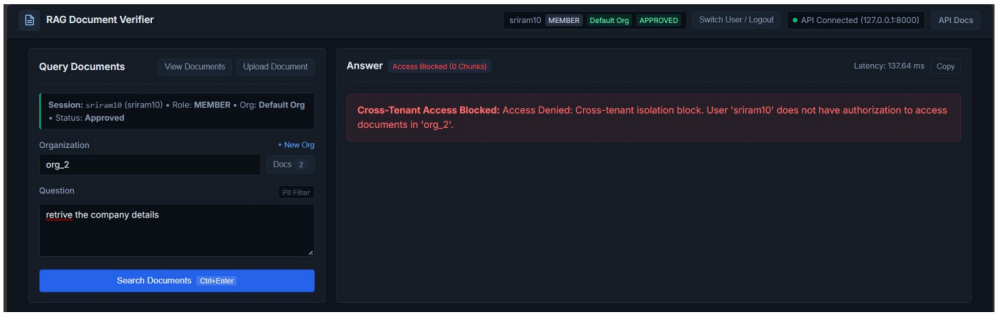


Table 8 Operational observations.

Figure	*Observation*	*Property shown*
5	Collections org_1 (7 points), org_2 (22), org_3, org_4 and org_10 (4 each), all green	Per-tenant collection isolation
6	Ciphertext, 96-bit nonce, SHA-256 hash and ACL stored; no plaintext	Payload confidentiality; ACL binding
6	Leaf hash, user and cloud proof, expected root stored with the point	Material for Gate 2
7a	User sriram20 (member, Org 2, approved) queries org_2: 22 chunks verified, 137.9 ms end to end	Authorised path
7b	User sriram10 (Default Org) queries org_2: blocked, 0 chunks, 137.64 ms end to end	Gate 1 tenant isolation

End-to-end times include retrieval, re-ranking and answer synthesis on the development machine and are not the verification-only costs of Table 5.

7 Discussion and limitations

The evaluation supports three conclusions. The checks that the client performs are inexpensive, at about 0.4 ms for 22 chunks, so verification is unlikely to dominate response time next to retrieval and generation. Each gate rejects the attack class assigned to it in function-level tests. And the DBV margin has a measurable effect on the required set, so λ should be tuned on the target corpus.

These results should be read with the limits of the prototype in mind, which Table 9 sets out together with the change that would remove each one. We think it more useful to state them than to leave readers to find them. The comparison with zero-knowledge systems is also limited: those systems prove that the whole retrieval was computed correctly, whereas this framework checks the authenticity of the returned payloads and the coverage of the agreed cells. Table 1 therefore lists reported costs for context and does not rank the systems.

Table 9 Limitations of the current prototype and planned remedies.

Limitation	*Observed behaviour*	*Planned remedy*
L1 Embeddings stored unmasked	Vectors are uploaded as generated, so inversion attacks apply (T2)	Add a secret orthogonal transform or noisy distance-comparison-preserving encryption (Ye et al., 2026); note known-plaintext risk
L2 No associated data	GCM is called without associated data; plaintext hash is not rechecked (A6)	Bind tenant and chunk identifiers as associated data and compare the decrypted hash with the leaf
L3 Root taken from the response	Gate 2 verifies against the root inside the proof (A7)	Verify against a client-held, versioned root with epoch numbers and consistency proofs
L4 Roots built per ingestion call	The organisation MMR is rebuilt from the chunks of the current ingestion call, not appended across the whole organisation	Persist the MMR and append new leaves, keeping earlier proofs valid
L5 Unauthenticated cell ids	Cell identifiers are payload fields outside the leaf; the default query path scans the whole collection, which satisfies coverage trivially	Commit the cell id in the leaf, require per-cell evidence and random spot checks
L6 Single key and default key	One AES key for all tenants, with a development default in the settings file	Per-tenant keys from a key-management service
L7 Authorisation after retrieval	RBAC is enforced in the client service after the cloud returns candidates	Collection-scoped credentials at the vector engine
L8 Heuristic grounding	CRAG and Self-RAG components are rule-based; lexical grounding is used without a neural encoder	Use trained relevance and support critics
L9 Small evaluation	188 chunks, five tenants, fallback embedder, single machine	Larger corpora, neural encoder, networked deployment

8 Conclusion

We presented a verification framework for RAG on an untrusted cloud vector engine that combines authenticated encryption, a two-tier Merkle mountain range, an IVF boundary check, tenant-level access control and grounding checks, with no server-side proving. In a working prototype the client-side checks for 22 retrieved chunks take 0.4 ms in-process, ciphertext tampering, forged proof elements, cell omission and cross-tenant access are rejected, and a margin ablation shows how the DBV factor trades required coverage against recall. The tests also show that the current prototype does not authenticate the binding between ciphertexts and chunks, does not anchor verification in a client-held root, and stores embeddings unmasked.

Future work will close these gaps in the order of Table 9: bind associated data and recheck hashes, anchor verification in a persistent versioned root, add per-cell evidence to DBV, mask embeddings, and repeat the evaluation with a neural encoder on larger multi-tenant corpora over a real network.

References

Agrawal, R., Kiernan, J., Srikant, R. and Xu, Y. (2004) 'Order-preserving encryption for numeric data', in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pp.563–574.

Asai, A., Wu, Z., Wang, Y., Sil, A. and Hajishirzi, H. (2024) 'Self-RAG: learning to retrieve, generate, and critique through self-reflection', in *Proceedings of the International Conference on Learning Representations (ICLR)*.

Cheon, J.H., Kim, A., Kim, M. and Song, Y. (2017) 'Homomorphic encryption for arithmetic of approximate numbers', in *Advances in Cryptology – ASIACRYPT*, pp.409–437.

Cormack, G.V., Clarke, C.L.A. and Büttcher, S. (2009) 'Reciprocal rank fusion outperforms Condorcet and individual rank learning methods', in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp.758–759.

Costan, V. and Devadas, S. (2016) 'Intel SGX explained', *IACR Cryptology ePrint Archive*, Report 2016/086.

Crosby, S.A. and Wallach, D.S. (2009) 'Efficient data structures for tamper-evident logging', in *Proceedings of the USENIX Security Symposium*, pp.317–334.

Dworkin, M. (2007) *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST Special Publication 800-38D, National Institute of Standards and Technology.

Gabizon, A., Williamson, Z.J. and Ciobotaru, O. (2019) 'PLONK: permutations over Lagrange-bases for oecumenical non-interactive arguments of knowledge', *IACR Cryptology ePrint Archive*, Report 2019/953.

Grislain, N. (2024) 'RAG with differential privacy', arXiv preprint arXiv:2412.19291.

Groth, J. (2016) 'On the size of pairing-based non-interactive zero-knowledge proofs', in *Advances in Cryptology – EUROCRYPT*, pp.305–326.

Jiang, Y., Yang, X., Liu, X., Guo, Y. and Zhang, J. (2026) 'zkRAG: efficiently proving RAG retrieval in zero knowledge', *IACR Cryptology ePrint Archive*, Report 2026/709.

Johnson, J., Douze, M. and Jégou, H. (2021) 'Billion-scale similarity search with GPUs', *IEEE Transactions on Big Data*, Vol. 7, No. 3, pp.535–547.

Lewis, P. et al. (2020) 'Retrieval-augmented generation for knowledge-intensive NLP tasks', in *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33, pp.9459–9474.

Li, X., Weng, C., Xu, Y., Wang, X. and Rogers, J. (2023) 'ZKSQL: verifiable and efficient query evaluation with zero-knowledge proofs', *Proceedings of the VLDB Endowment*, Vol. 16, No. 8, pp.1804–1816.

Li, Y. et al. (2025) 'Generating is believing: membership inference attacks against retrieval-augmented generation', in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, arXiv:2406.19234.

Lin, C., Cui, Y., Zhou, Z., Hong, C., Wang, Y., Chen, Z. and Li, M. (2026) 'VeriRAG: efficient zero-knowledge proofs for verifiable retrieval-augmented generation', *IACR Cryptology ePrint Archive*, Report 2026/637.

Malkov, Y.A. and Yashunin, D.A. (2020) 'Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs', *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Vol. 42, No. 4, pp.824–836.

Merkle, R.C. (1987) 'A digital signature based on a conventional encryption function', in *Advances in Cryptology – CRYPTO '87*, Lecture Notes in Computer Science, Vol. 293, pp.369–378.

Moriarty, K., Kaliski, B. and Rusch, A. (2017) *PKCS #5: Password-Based Cryptography Specification Version 2.1*, RFC 8018, Internet Engineering Task Force.

Morris, J.X. et al. (2023) 'Text embeddings reveal (almost) as much as text', in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp.12448–12460.

NIST (2015) *Secure Hash Standard (SHS)*, FIPS PUB 180-4, National Institute of Standards and Technology.

Palanisamy, B., Chalapathi, G.S.S., Hassija, V. and Buyya, R. (2026) 'Security and privacy in retrieval-augmented generation: architectures, threats, defenses, and future directions for building trustworthy systems', arXiv preprint arXiv:2606.25533.

Qiu, Z., Qu, W., Zhang, J. and Yuan, B. (2026) 'V3DB: audit-on-demand zero-knowledge proofs for verifiable vector search over committed snapshots', arXiv preprint arXiv:2603.03065.

Reimers, N. and Gurevych, I. (2019) 'Sentence-BERT: sentence embeddings using Siamese BERT-networks', in *Proceedings of EMNLP-IJCNLP*, pp.3982–3992.

Robertson, S. and Zaragoza, H. (2009) 'The probabilistic relevance framework: BM25 and beyond', *Foundations and Trends in Information Retrieval*, Vol. 3, No. 4, pp.333–389.

Todd, P. (2018) *Merkle Mountain Ranges*, GitHub repository specification [online] https://github.com/opentimestamps/opentimestamps-server/blob/master/doc/merkle-mountain-range.md.

Wang, W., Wei, F., Dong, L., Bao, H., Yang, N. and Zhou, M. (2020) 'MiniLM: deep self-attention distillation for task-agnostic compression of pre-trained transformers', in *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33, pp.5776–5788.

Wu, H. et al. (2025) 'Zero-knowledge verifiable graph query evaluation via expansion-centric operator decomposition', arXiv preprint arXiv:2507.00427.

Yan, S.Q. et al. (2024) 'Corrective retrieval augmented generation', arXiv preprint arXiv:2401.15884.

Ye, H., Guo, J., Liu, Z. and Lam, K.-Y. (2026) 'ppRAG: efficient privacy-preserving retrieval-augmented generation with distance-preserving encryption', arXiv preprint arXiv:2601.12331.

Zhang, Z. et al. (2026) 'VERIF: an efficient zero-knowledge proof system for verifying IVF-Flat retrieval in RAG services', *IACR Cryptology ePrint Archive*, Report 2026/1802.